PRACTICAL ASSESSMENT / FRONTEND DEVELOPMENT

Build an Event Enrollment Manager

React.js coding assessment • 4 hours • 100 marks • Individual submission

DURATION	TECHNOLOGY	DELIVERABLE
4 hours	React.js + Vite	Working application + README

01 | Problem statement

A community learning centre runs short events on technology, creativity, and life skills. It currently manages enrollments manually. Your task is to develop a **responsive React web application** that lets visitors browse upcoming events, register for an available seat, view existing registrations, and cancel a registration.

Build the frontend only. Use the supplied event data as your starting dataset and store registrations in the browser. **No backend, authentication, or external API is required.**

02 | Required functionality

A. Browse events (20 marks)

- Display all six supplied events as readable cards or rows. Show title, category, date, time, total seats and available seats.
- Provide a search box to filter by event title, ignoring letter case.
- Provide a category filter with **All, Technology, Arts, Life Skills**.
- Display a clear empty-state message when no event matches the search/filter.

B. Register for an event (25 marks)

- Each event must provide a **Register** action that opens or displays a form for the selected event.
- Collect the attendee's **full name** and **email address**; both are mandatory.
- Validate required fields and basic email format. Show useful validation messages.
- Prevent registration if the event has no available seats.
- Prevent the **same email address** from registering twice for the **same event** (trim spaces and compare without letter case). The email may register for other events.
- After successful registration, show confirmation and immediately update the available seat count.

C. View and cancel registrations (15 marks)

- Show all registrations with attendee name, email, event title and event date.
- Provide a **Cancel** action for each registration. Confirm before cancellation and restore the event's available-seat count.

03 | Application behavior and technical rules

D. Navigation and user interface (10 marks)

- Provide two routes/pages using **React Router**: **Events** and **Registrations**.
- Include clear navigation and a readable layout that works on desktop and mobile screens.
- Disable or clearly label registration for events that are fully booked.

E. React implementation (15 marks)

- Build the application with reusable functional components, JSX and props.
- Use React state and event handlers for user interaction; use appropriate hooks (such as **useState** and **useEffect**).
- Render collections with stable keys; update state without directly mutating arrays or objects.

F. Persistence and stability (10 marks)

- Save registrations to **localStorage** so that they remain available after refreshing the page.
- Calculate seat availability from event capacity minus active registrations. Changes must remain correct after reload.
- Handle normal empty states without crashes or uncaught console errors.

04 | Starter event data

Use these six events as your initial fixed dataset. Dates and times are displayed in local time; the centre starts with zero registrations. You may store the data in a JavaScript file.

ID	Event title	Category	Date	Time	Seats
EV101	Intro to Cloud Computing	Technology	10 Nov 2026	10:00	4
EV102	Creative Writing Lab	Arts	12 Nov 2026	14:00	3
EV103	Data Visualization Basics	Technology	14 Nov 2026	11:00	5
EV104	Personal Finance Essentials	Life Skills	16 Nov 2026	16:00	4
EV105	Public Speaking Practice	Life Skills	18 Nov 2026	10:30	3
EV106	Digital Illustration Studio	Arts	20 Nov 2026	13:00	2

05 | Scope and constraints

- **Required:** React.js, Vite, React Router, browser localStorage, and CSS (plain CSS or a CSS framework).
- **Not required:** backend services, APIs, SQL, authentication, admin dashboard, or Docker.
- You may choose your component structure and UI design. Additional features do not replace any required feature.

Important rule: Capacity is the maximum number of active registrations per event. A canceled registration does not occupy a seat.

06 | Acceptance checks

Your assessor should be able to verify the following behaviors using the finished app:

01. On first launch, exactly six events are visible and their available seats equal their capacities.

02. Selecting **Arts** shows two events. Searching for **visual** matches “Data Visualization Basics”.

03. A valid registration succeeds, appears on Registrations, and decreases the event’s available seats by one.

04. Missing name, missing email, or an invalid email produces a validation message; no registration is created.

05. The same email cannot be registered again for the same event, including letter-case variations; it can be used for another event.

06. A fully booked event rejects further registrations.

07. Canceling a registration removes it from the active list and restores one available seat.

08. Refreshing the browser keeps registrations and the resulting seat counts.

09. Both routes work correctly and an empty search result displays a helpful message.

07 | Evaluation rubric

Assessment area	Marks
Event listing, search and category filter	20
Registration form, validation and capacity rules	25
Registration list and cancellation	15
Routing, navigation and responsive UI	10
React component structure and state handling	15
localStorage persistence and stability	10
README and submission completeness	5
Total	**100**

08 | Submission instructions

- Submit a ZIP file of your React project named **React_Event_Enrollment_YourName.zip**.
- Include source code, event data, configuration files, and a **README.md** with installation and start instructions (including required npm commands).
- Do **not** include **node_modules** in the ZIP. The project must run after dependency installation.
- Demonstrate the application locally to the assessor. Be prepared to explain your components, state updates and validation.

Time limit: 4 hours from the start of the assessment. Submit the working version within the allotted time.